Specification for Dead-Code Elimination and Copy Propagation on Lowered Eta IR CFGs

1 Overview

This document specifies several optimizations on the lowered Eta IR: *dead-code elimination* (DCE), *copy propagation* (CP) and *control flow elimination* (CFE) The input language is the lowered form of the IR in which

(i) expressions of the form $(\sigma; e)$ do not occur,

(ii) expression calls `call` $e_0(e_1, \ldots, e_n)$ do not occur,

(iii) each function body is a single top-level sequence statement,

(iv) each conditional jump statement has only a true label, and passes control to the consecutive sequence if false,

(v) control flow is represented only with labels, jumps, conditional jumps, statement calls, and returns.

Because lowered IR expressions contain neither ESEQ nodes nor call expressions, evaluating an expression does not itself modify memory or control flow. This fact is used by DCE.

The optimizations are defined over a control-flow graph (CFG) whose vertices are *individual IR statements*, not basic blocks. The pipeline for one function is

$$\text{lowered IR} \longrightarrow \text{statement-level CFG} \longrightarrow \text{analysis and rewrite} \longrightarrow \text{lowered IR}.$$

The optimizations are defined independently. An implementation may run either pass alone, or run CP followed by DCE and iterate to a fixed point.

2 Lowered IR and Auxiliary Notation

2.1 Grammar of lowered IR

For this pass, a function body is a single sequence

$$\sigma = s_1 \; s_2 \; \cdots \; s_n.$$

Each statement is one of the following:

$$s ::= d \leftarrow e \mid \texttt{jump}\ e \mid \texttt{cjump}\ e\ L_t \mid \texttt{call}^k\ e_0(e_1, \ldots, e_m) \mid \texttt{return}\ e_1, \ldots, e_r \mid L :$$

where k is the statically known return arity of the call site. The destination grammar is

$$d ::= t \mid [e]$$

and the lowered expressions are

$$e ::= n \mid t \mid e_1 \ op \ e_2 \mid [e] \mid \&L.$$

Here t ranges over temporaries and L ranges over labels.

Important difference of our cfg with the one shown in the course Our cfg includes jumps and labels instead of eliminating them. This is for the convinience of transerring cfg back to IR language so that we do not have to reassign labels and jumps at the nodes where control flow merges/seperates. We can simply extract the cfg nodes in their original order and do not have to consider the control flow relationship, which would be more detailedly illustrated in "Transform from Optimized CFG Back to Lowered IR" part.

Lowered IR properties The input to these passes is assumed to satisfy the following properties:

(1) labels occur only as statements,

(2) the body is flat (no nested sequence nodes),

(3) every transfer of control is explicit via `jump`, `cjump`, or `return`

(4) conditional jump in lowered IR only has the if branch which jumps when condition is true, false condition falls to the next statement

2.2 Temporaries used by an expression

Define $\mathsf{Temps}(e)$ recursively by

$$\begin{aligned}
\mathsf{Temps}(n) &= \varnothing, \\
\mathsf{Temps}(t) &= \{t\}, \\
\mathsf{Temps}(\&L) &= \varnothing, \\
\mathsf{Temps}([e]) &= \mathsf{Temps}(e), \\
\mathsf{Temps}(e_1 \ op \ e_2) &= \mathsf{Temps}(e_1) \cup \mathsf{Temps}(e_2).
\end{aligned}$$

If the distinguished ABI temporaries $\mathsf{RV}_1, \mathsf{RV}_2, \ldots$ occur explicitly in the IR, they are included in $\mathsf{Temps}(e)$ exactly like ordinary temporaries.

3 Control-Flow Graph Data Structure

3.1 Statement nodes

We evaluates the CFG from the body of each function declaration in the lowered IR explicitly. A CFG node represents exactly one lowered IR statement, except for one synthetic start node.

$$\begin{aligned}
G &::= [n_0, n_1, n_2, ..., n_m] \\
n &::= (\sigma, pred, n_{next}, n_{if}) \\
pred &::= \{n_1, ..., n_p\}
\end{aligned}$$

Define function:

$$label2node(L)$$

where:

- G is a finite set of nodes, with n_0 being the entry node.
- Every node $n_k, k > 0$ has exactly one associated IR statement σ.
- Every node n_k has a list of predecessor nodes *pred*. When $k = 0$, which means n_k is a successor node, $pred = \phi$.
- *label2node* maps every label L occurring in the function to the unique node whose statement is L :.
- n_{next} points to the direct successor of the node. If $\sigma = jump$, points to *label2node*(L)instead. No successor for $\sigma = jump$.
- n_{if} points to *label2node*(L) for $\sigma = cjump$. Null for other nodes.

4 Transform from Lowered IR to CFG

4.1 Node creation

Create one synthetic entry node $n_0 = \mathsf{entry}$. For each statement s_i with $1 \leq i \leq m$, create a distinct node n_i such that

$$\sigma(n_i) = s_i,$$

Then

$$G = \{n_0, n_1, \ldots, n_m\}.$$

Construct the map *label2node* by scanning all statement nodes and, for each node n_i with statement L :, setting

$$label2node(L) = n_i.$$

Connect the points Notice that we haven't connected the points which modifies the saved-properties of every node. Thus we define a *reconnection* function to do this job.
First, for all $n_i \in G$, we clear all their previous connection by setting:

$$n_{next}(n_i) = \text{null}$$

$$n_{if}(n_i) = \text{null}$$

$$pred(n_i) = \varnothing$$

Then connect the nodes:

$$n_{next}(n_i) = \begin{cases} label2node(L) & \sigma(n_i) = jump \\ \text{null} & i = m | \sigma(n_i) = return \\ n_{i+1} & \text{otherwise} \end{cases}$$

$$n_{if}(n_i) = \begin{cases} label2node(L) & \sigma(n_i) = cjump \\ \text{null} & \text{otherwise} \end{cases}$$

$$\begin{cases} pred(label2node(L)) += n_i, pred(n_{i+1}) += n_i & \sigma(n_i) = cjump \\ pred(label2node(L)) += n_i & \sigma(n_i) = jump \\ \text{Do nothing} & i = m | \sigma(n_i) = return \\ pred(n_{i+1}) += n_i & \text{otherwise} \end{cases}$$

The reason why we define this process as an independent function because every time the set of nodes in cfg changed after an optimization process, the connection need to be re-established to avoid connecting a node in cfg to a previously deleted node. So we would call *reconnection* everytime we modified the set of nodes in cfg.

5 Transform from Optimized CFG Back to Lowered IR

After optimization, the CFG is converted back to a single sequence statement. All the rewrites and eliminations in this specification never reorder surviving nodes, never create new labels, and never change the target labels of existing jumps. Copy propagation only rewrites expressions inside nodes. Dead-code elimination only deletes certain temporary-assignment nodes and reconnects control flow around them. Therefore every surviving real node still has its original order index. The optimized and semantically-preserving statement sequence can be obtained by simply keeping the sequence of the vector of nodes in G.

More explicitly, let

$$G = [n_0, n_1, \ldots, n_m]$$

in which n_0 is start node which contains no statement.

The reconstructed lowered IR body is the single sequence

$$\sigma(n_1)\ \sigma(n_2)\ \cdots\ \sigma(n_m).$$

No additional synthetic statements are emitted.

6 Reachability cleanup

Let

$$\mathsf{Reach}(G) = \text{the set of nodes reachable from } \mathsf{entry} \text{ by following } \mathsf{succ}.$$

A cleanup pass may replace G by the induced subgraph on $\mathsf{Reach}(G)$. The process can be realized by a simple DFS from the starting node.

7 Live-Variable Analysis for Dead-Code Elimination

7.1 Statement-level use/def sets

For each real statement node $n \neq \mathsf{entry}$, define $\mathsf{use}(n)$ and $\mathsf{def}(n)$ from $\mathsf{stmt}(n)$ by:

$\mathsf{stmt}(n)$	$\mathsf{use}(n)$	$\mathsf{def}(n)$
$x \leftarrow e$	$\mathsf{Temps}(e)$	$\{x\}$
$[e_1] \leftarrow e_2$	$\mathsf{Temps}(e_1) \cup \mathsf{Temps}(e_2)$	$\varnothing$
$\texttt{call}^k\ e_0(e_1, \ldots, e_n)$	$\bigcup_{i=1}^{n} \mathsf{Temps}(e_i)$	$\{_\mathsf{RV}_1, \ldots, _\mathsf{RV}_k\}$
$\texttt{jump}\ e$	$\varnothing$	$\varnothing$
$\texttt{cjump}\ e\ L_t\ L_f$	$\mathsf{Temps}(e)$	$\varnothing$
$\texttt{return}\ e_1, \ldots, e_r$	$\bigcup_{i=1}^{r} \mathsf{Temps}(e_i)$	$\varnothing$
$L:$	$\varnothing$	$\varnothing$

For the synthetic entry node,

$$\mathsf{use}(\mathsf{entry}) = \varnothing, \qquad \mathsf{def}(\mathsf{entry}) = \varnothing.$$

The treatment of `call` reflects that a call site with return arity k overwrites the ABI return value temporaries $_\mathsf{RV}_1, \ldots, _\mathsf{RV}_k$.

7.2 Dataflow equations

Live-variable analysis is a backward may-analysis. For every node $n \in \mathsf{Reach}(G)$,

$$\mathsf{out}(n) = \bigcup_{s \in \{n_{next}(n)\} \cup \{n_{if}(n)\}} \mathsf{in}(s)$$

and

$$\mathsf{in}(n) = \mathsf{use}(n) \cup (\mathsf{out}(n) - \mathsf{def}(n)).$$

If $\mathsf{succ}(n) = \varnothing$, then $\mathsf{out}(n) = \varnothing$.

7.3 Detailed worklist algorithm

The worklist algorithm for liveness is:

1. For every reachable node n, initialize

$$\mathsf{in}(n) := \varnothing, \qquad \mathsf{out}(n) := \varnothing.$$

2. Initialize a FIFO worklist with all reachable nodes.

3. While the worklist is nonempty:

   (a) remove one node n from the worklist;

   (b) compute

$$\mathsf{out}_{\text{new}}(n) := \bigcup_{s \in \{n_{next}(n)\} \cup \{n_{if}(n)\}} \mathsf{in}(s);$$

(c) compute

$$\mathsf{in}_{\text{new}}(n) := \mathsf{use}(n) \cup (\mathsf{out}_{\text{new}}(n) - \mathsf{def}(n));$$

(d) if either $\mathsf{in}_{\text{new}}(n) \neq \mathsf{in}(n)$ or $\mathsf{out}_{\text{new}}(n) \neq \mathsf{out}(n)$, then:

(i) set $\mathsf{in}(n) := \mathsf{in}_{\text{new}}(n)$ and $\mathsf{out}(n) := \mathsf{out}_{\text{new}}(n)$;

(ii) for every $p \in \mathsf{pred}(n)$, add p to the worklist if it is not already present.

4. When the worklist becomes empty, the fixed point has been reached.

7.4 Which nodes are dead

A node is removable by DCE exactly when all of the following hold:

(1) $\sigma(n_i)$ has the form $x \leftarrow e$ for a temporary x;

(2) $x \notin \mathsf{out}(n)$; and

(3) $i \neq 0$.

No other statement form is removed by this pass. In particular:

- a `call` node is never removed, because it may have side effects and may change memory;
- a memory store node $[e_1] \leftarrow e_2$ is never removed, because it modifies memory;
- label, `jump`, `cjump`, and `return` nodes are never removed.

Because lowered expressions contain neither ESEQ nor call expressions, the right-hand side e of $x \leftarrow e$ is side-effect free. Therefore deleting such a node is semantics-preserving whenever x is dead immediately after that node.

8 Available-Copy Analysis for Copy Propagation

8.1 Copy facts and lattice

Let the universe of copy facts be

$$\mathsf{Copies} = \{x \rightsquigarrow y \mid x, y \text{ are temporaries and } x \neq y\}.$$

The intended meaning of $x \rightsquigarrow y$ at a program point is:

> Along every path to this program point, the current value of x is exactly the current value of y.

This is a forward must-analysis. Therefore the meet operator is set intersection.

8.2 Node transfer information

For any node n, define

$$\mathsf{kill}(n) = \{u \rightsquigarrow v \in \mathsf{Copies} \mid u \in \mathsf{def}(n) \text{ or } v \in \mathsf{def}(n)\}.$$

The generation function is

$$\mathsf{gen}(n) = \begin{cases} \{x \rightsquigarrow y\} & \text{if } \mathsf{stmt}(n) \text{ is } x \leftarrow y \text{ with } x \neq y, \\ \varnothing & \text{otherwise.} \end{cases}$$

Thus only a direct temporary-to-temporary copy generates a copy fact. The transfer function of one node is

$$\mathsf{F}_n(A) = \mathsf{gen}(n) \cup (A - \mathsf{kill}(n)).$$

8.3 Dataflow equations

For the synthetic entry node,

$$\mathsf{in}(\mathsf{entry}) = \varnothing, \qquad \mathsf{out}(\mathsf{entry}) = \varnothing.$$

For every reachable real node $n \neq \mathsf{entry}$,

$$\mathsf{in}(n) = \bigcap_{p \in \mathsf{pred}(n)} \mathsf{out}(p)$$

and

$$\mathsf{out}(n) = \mathsf{F}_n(\mathsf{in}(n)).$$

If a reachable real node somehow has no predecessors other than after earlier inconsistencies, it is not on any entry-reachable path and should be removed by the reachability cleanup before analysis.

8.4 Detailed worklist algorithm

The worklist algorithm for available copies is:

1. Initialize
$$\mathsf{in}(\mathsf{entry}) := \varnothing, \qquad \mathsf{out}(\mathsf{entry}) := \varnothing.$$
For every reachable real node $n \neq \mathsf{entry}$, initialize
$$\mathsf{in}(n) := \mathsf{Copies}, \qquad \mathsf{out}(n) := \mathsf{Copies}.$$
2. Initialize a FIFO worklist with all reachable nodes.
3. While the worklist is nonempty:
   (a) remove one node n from the worklist;
   (b) compute
$$\mathsf{in}_{\mathrm{new}}(n) := \begin{cases} \varnothing & \text{if } n = \mathsf{entry}, \\ \bigcap_{p \in \mathsf{pred}(n)} \mathsf{out}(p) & \text{otherwise;} \end{cases}$$

(c) compute

$$\mathsf{out}_{\mathrm{new}}(n) := \begin{cases} \varnothing & \text{if } n = \mathsf{entry}, \\ \mathsf{F}_n(\mathsf{in}_{\mathrm{new}}(n)) & \text{otherwise;} \end{cases}$$

(d) if either $\mathsf{in}_{\mathrm{new}}(n) \neq \mathsf{in}(n)$ or $\mathsf{out}_{\mathrm{new}}(n) \neq \mathsf{out}(n)$, then:

(i) set $\mathsf{in}(n) := \mathsf{in}_{\mathrm{new}}(n)$ and $\mathsf{out}(n) := \mathsf{out}_{\mathrm{new}}(n)$;

(ii) for every $s \in \mathsf{succ}(n)$, add s to the worklist if it is not already present.

4. When the worklist becomes empty, the fixed point has been reached.

8.5 Canonical replacement function

Given a fact set $A \subseteq \mathsf{Copies}$, define the canonical replacement of a temporary by

$$\mathsf{canon}_A(t) = \begin{cases} \mathsf{canon}_A(u) & \text{if } t \rightsquigarrow u \in A, \\ t & \text{otherwise.} \end{cases}$$

Because any definition of a temporary kills all copy facts mentioning that temporary, at a fixed program point there can be at most one outgoing fact $t \rightsquigarrow u$ for a given left-hand side t. Hence canon_A is well-defined. Operationally, one may compute it by repeatedly following copy links until a temporary with no outgoing copy fact is reached.

8.6 Expression rewriting

Given a fact set A, define expression rewriting recursively:

$$\begin{aligned} \mathsf{rewrite}_A(n) &= n, \\ \mathsf{rewrite}_A(t) &= \mathsf{canon}_A(t), \\ \mathsf{rewrite}_A(\&L) &= \&L, \\ \mathsf{rewrite}_A([e]) &= [\mathsf{rewrite}_A(e)], \\ \mathsf{rewrite}_A(e_1 \ op \ e_2) &= \mathsf{rewrite}_A(e_1) \ op \ \mathsf{rewrite}_A(e_2). \end{aligned}$$

Using this expression rewrite, statement rewriting at a node is defined by replacing temporaries only in use positions:

$$\begin{aligned} \mathsf{rewrite}_A(x \leftarrow e) &= x \leftarrow \mathsf{rewrite}_A(e), \\ \mathsf{rewrite}_A([e_1] \leftarrow e_2) &= [\mathsf{rewrite}_A(e_1)] \leftarrow \mathsf{rewrite}_A(e_2), \\ \mathsf{rewrite}_A(\texttt{call}^k \ e_0(e_1, \ldots, e_n)) &= \texttt{call}^k \ \mathsf{rewrite}_A(e_0)(\mathsf{rewrite}_A(e_1), \ldots, \mathsf{rewrite}_A(e_n)), \\ \mathsf{rewrite}_A(\texttt{jump} \ e) &= \texttt{jump} \ \mathsf{rewrite}_A(e), \\ \mathsf{rewrite}_A(\texttt{cjump} \ e \ L_t \ L_f) &= \texttt{cjump} \ \mathsf{rewrite}_A(e) \ L_t \ L_f, \\ \mathsf{rewrite}_A(\texttt{return} \ e_1, \ldots, e_r) &= \texttt{return} \ \mathsf{rewrite}_A(e_1), \ldots, \mathsf{rewrite}_A(e_r), \\ \mathsf{rewrite}_A(L :) &= L : . \end{aligned}$$

The left-hand side temporary of $x \leftarrow e$ is *not* rewritten, because it denotes a new definition site.

8.7 Actual CFG transform for copy propagation

For each reachable real node $n \neq \mathsf{entry}$:

1. Let $A := \mathsf{in}(n)$.
2. Replace $\mathsf{stmt}(n)$ by the rewritten statement
$$\mathsf{stmt}'(n) := \mathsf{rewrite}_A(\mathsf{stmt}(n)).$$
3. If the rewritten statement is of the form $x \leftarrow x$, mark n for deletion as a trivial self-copy.

After rewriting all reachable real nodes, delete every node marked as a trivial self-copy using the same graph-deletion procedure as in DCE.

Effect on control flow. Pure copy propagation does not change jump targets or branch structure. Apart from removing trivial self-copy nodes, it leaves the CFG shape unchanged.

9 Local Value Numbering

9.1 Purpose and scope

Local value numbering (LVN) is a local expression-rewrite transform. It scans the CFG node vector from left to right and maintains a value-numbering table for the current local block. When a binary expression has already been computed in the current block and some temporary is still recorded as holding that value number, the expression is rewritten to that temporary.

This implementation does not run a global dataflow fixed point for value numbering. It does not propagate value-numbering facts through labels or across control-flow terminators. It also does not delete nodes. The pass only rewrites expressions inside existing CFG nodes.

9.2 Local blocks in a statement-level CFG

The implementation operates directly on the statement-level node vector

$$G = [n_0, n_1, \ldots, n_m].$$

It does not allocate separate basic-block objects. Instead, it carries one table while scanning consecutive nodes. A fresh empty table is created

(1) at the beginning of the scan,

(2) at every label node, and

(3) immediately after a `jump`, `cjump`, or `return` node has been processed.

Thus a local LVN block is the maximal consecutive portion of the node vector between these reset points. Labels are always block boundaries in the actual implementation, even if a label has only a fall-through predecessor.

9.3 Value-numbering tables

Within one local block, the pass maintains the following mutable table:

tempVN	maps each currently available temporary to its value number,
vnTemps	maps each value number to the set of temporaries currently holding it,
vnExprVN	maps binary-expression keys to value numbers,
constVN	maps integer constants to value numbers,
memVN	maps numbered memory-address expressions to memory-load value numbers,
addressVN	maps label addresses to value numbers,
currentValue	is the next fresh value number.

Fresh value numbers are unsigned integers starting at 0 in each local block.

9.4 Expression numbering and rewriting

The procedure

$$\mathsf{vnKeyRewrite}(e, A) = (v, e')$$

updates table A, returns the value number v of expression e, and returns the rewritten expression e'.

Constants. For an integer literal z, reuse $\mathsf{constVN}(z)$ if it exists. Otherwise assign z the next fresh value number. The rewritten expression is the same integer literal.

Temporaries. For a temporary t, reuse $\mathsf{tempVN}(t)$ if it exists. Otherwise assign t the next fresh value number and initialize $\mathsf{vnTemps}(v) = \{t\}$. The rewritten expression is the same temporary t.

Label addresses. For an address $\&L$, reuse $\mathsf{addressVN}(L)$ if it exists. Otherwise assign L the next fresh value number. The rewritten expression is the same address expression.

Binary expressions. For $e_1\ op\ e_2$, first compute

$$\mathsf{vnKeyRewrite}(e_1, A) = (v_1, e_1'), \qquad \mathsf{vnKeyRewrite}(e_2, A) = (v_2, e_2').$$

Construct the key

$$K = (v_1, v_2, op).$$

The implementation keeps the original left/right order in this key; it does not sort operands for commutative operators. If $\mathsf{vnExprVN}(K) = v$ exists and $\mathsf{vnTemps}(v)$ is nonempty, the expression is rewritten to an arbitrary temporary from $\mathsf{vnTemps}(v)$. If the key exists but no temporary currently holds v, the expression is rebuilt as $e_1'\ op\ e_2'$. If the key is new, assign it the next fresh value number and return the rebuilt binary expression.

Memory loads. For a memory load $[e]$, first compute $\mathsf{vnKeyRewrite}(e, A) = (v_e, e')$. The implementation uses v_e as the key into memVN. If that key exists, the load gets the recorded value number; otherwise the key is assigned the next fresh value number. The rewritten expression is $[e']$.

Disallowed expression forms. Function-call expressions and expression-sequence expressions are not expected after lowering. If one is encountered during value numbering, the implementation raises an error.

9.5 Defining temporaries

When a statement defines a temporary x to have value number v, the implementation updates the tables as follows:

1. If x already has an old value number $v_{\text{old}} = \mathsf{tempVN}(x)$, remove x from $\mathsf{vnTemps}(v_{\text{old}})$.
2. Set $\mathsf{tempVN}(x) := v$.
3. Add x to $\mathsf{vnTemps}(v)$.

The right-hand side is numbered before this update, so a statement such as $x \leftarrow x + 1$ reads the old value of x and only then assigns a new value number to x.

9.6 Statement rewrite rules

LVN scans each local block from left to right. At each reset point, the value-numbering table is empty.

Temporary assignment. For

$$x \leftarrow e,$$

compute $\mathsf{vnKeyRewrite}(e, A) = (v, e')$, rewrite the node to

$$x \leftarrow e',$$

and then define x to have value number v.

Memory store. For

$$[e_1] \leftarrow e_2,$$

compute

$$\mathsf{vnKeyRewrite}(e_1, A) = (v_1, e_1'), \qquad \mathsf{vnKeyRewrite}(e_2, A) = (v_2, e_2'),$$

rewrite the node to

$$[e_1'] \leftarrow e_2',$$

and leave the table otherwise unchanged.

Call. For

$$\texttt{call}^k\ e_0(e_1, \ldots, e_m),$$

rewrite each argument expression with $\mathsf{vnKeyRewrite}$:

$$\texttt{call}^k\ e_0(e_1', \ldots, e_m').$$

The function expression e_0 is not rewritten by the current implementation. Then, for each ABI return temporary $_\mathsf{RV}_i$ with $1 \le i \le k$, remove any old association for $_\mathsf{RV}_i$, assign it the next fresh value number, and set $\mathsf{vnTemps}(v_i) = \{_\mathsf{RV}_i\}$.

Control-flow and return statements. For

$$\texttt{cjump}\ e\ L_t, \qquad \texttt{return}\ e_1, \ldots, e_r,$$

rewrite the expression uses with vnKeyRewrite. A `jump` statement and a label statement are not rewritten by LVN. A `jump`, `cjump`, or `return` ends the current local block.

9.7 Actual CFG transform for local value numbering

The full transform is:

1. Initialize an empty value-numbering table and set a flag indicating that the next node starts a fresh local block.
2. Scan the node vector G from left to right, including the synthetic entry node.
3. Before processing a node, if the fresh-block flag is set or the node is a label, replace the current table with a new empty table.
4. Apply the statement rewrite rule for the node, producing the table for the following node.
5. Set the fresh-block flag exactly when the node is a `jump`, `cjump`, or `return`.

The transform does not delete nodes, does not reconnect the CFG, and does not change the order of the node vector.

9.8 Example

Consider the following straight-line region:

$$\begin{aligned} a &\leftarrow b + c \\ d &\leftarrow b + c \\ e &\leftarrow d * 2 \\ b &\leftarrow 0 \\ f &\leftarrow c + b \end{aligned}$$

The first expression gives $b + c$ a value number and records a as a temporary holding that value. When the second statement computes the same ordered expression key, LVN may rewrite it as

$$d \leftarrow a.$$

The implementation does not canonicalize commutative operands, so the expression $c + b$ has a different key from $b + c$. After $b \leftarrow 0$, the old value number for b is no longer associated with temporary b.

10 Conditional Constant Propagation

10.1 Constant facts and lattice

Conditional constant propagation (CCP) is a forward analysis that combines ordinary constant propagation with executable-edge discovery. Unlike the available-copy analysis above, it does not assume that every syntactic CFG edge is feasible. Instead, when the condition of a reachable `cjump`

is known to be constant, only the feasible successor edge is marked executable. This allows the pass to discover unreachable statements and to simplify later control-flow cleanup.

We would not implement the MOP in our CCP because tracking all complete paths in a CFG can be expensive, especially when there're numerous branches and loops which would cause the number of paths to increase exponentially, undermining the performance of the compiler.

Let the per-temporary constant-value lattice be

$$\mathcal{L}_c = \{\mathsf{NAC}\} \cup \{\mathsf{Const}(z) \mid z \in \mathbb{Z}\} \cup \{\mathsf{Undef}\}.$$

The intended meanings are:

- Undef: no executable path has yet supplied a value for this temporary;
- $\mathsf{Const}(z)$: every currently executable path supplies the integer constant z;
- NAC: the value is not a compile-time constant.

The order is

$$\mathsf{NAC} \sqsubseteq \mathsf{Const}(z) \sqsubseteq \mathsf{Undef},$$

with different constants incomparable. The meet operator $\sqcap$ is

$$\begin{aligned} \mathsf{Undef} \sqcap v &= v, \\ \mathsf{NAC} \sqcap v &= \mathsf{NAC}, \\ \mathsf{Const}(z) \sqcap \mathsf{Const}(z) &= \mathsf{Const}(z), \\ \mathsf{Const}(z_1) \sqcap \mathsf{Const}(z_2) &= \mathsf{NAC} \qquad (z_1 \neq z_2). \end{aligned}$$

Also, we keep

$$U(n) \in \{\mathsf{true}, \mathsf{false}\}$$

for each node to track whether the node is unreachable under the current constant propagation (true means unreachable). The meet operator $\sqcap = \wedge$.

Thus, we define the dataflow object to be:

$$C(n) = (U(n),\, \rho(n))$$

The meet operator $\sqcap$ is defined as:

$$\sqcap(C(n_1)...C(n_m)) = (\sqcap(U(n_1)...U(n_m)),\, \sqcap\rho(n_k)) \qquad U(n_k) == \mathsf{false}$$

10.2 Expression evaluation over the constant lattice

For an environment ρ, define abstract evaluation

$$[\![e]\!]_\rho \in \mathcal{L}_c$$

as follows:

$$\begin{aligned} [\![z]\!]_\rho &= \mathsf{Const}(z), \\ [\![t]\!]_\rho &= \rho(t), \\ [\![\&L]\!]_\rho &= \mathsf{NAC}, \\ [\![[e]]\!]_\rho &= \mathsf{NAC}. \end{aligned}$$

A memory load is conservatively treated as NAC because this pass does not track memory contents. Address expressions inside loads may still be rewritten later if they contain constant temporaries.

For a binary expression, abstract evaluation is

$$[\![e_1 \ op \ e_2]\!]_\rho = \mathsf{evalop}\big(op, [\![e_1]\!]_\rho, [\![e_2]\!]_\rho\big),$$

where

$$\mathsf{evalop}(op, v_1, v_2) = \begin{cases} \mathsf{Const}(z_1 \ op \ z_2) & v_1 = \mathsf{Const}(z_1),\ v_2 = \mathsf{Const}(z_2), \text{and the operation is defined,} \\ \mathsf{NAC} & v_1 = \mathsf{NAC} \text{ or } v_2 = \mathsf{NAC}, \\ \mathsf{NAC} & v_1 = \mathsf{Const}(z_1),\ v_2 = \mathsf{Const}(z_2), \text{but the operation is not safe to fold,} \\ \mathsf{Undef} & \text{otherwise.} \end{cases}$$

For example, division or remainder by a constant zero is not folded and is treated as NAC.

10.3 Transfer function

For a node n, let $\rho[x \mapsto v]$ denote the environment equal to ρ except that x maps to v. The constant-propagation transfer function for constant environment ρ is

$$\mathsf{CTransfer}_n(\rho(n)) = \begin{cases} \rho[x \mapsto [\![e]\!]_\rho] & \mathsf{stmt}(n) = x \leftarrow e, \\ \rho[_\mathsf{RV}_1 \mapsto \mathsf{NAC}, \ldots, _\mathsf{RV}_k \mapsto \mathsf{NAC}] & \mathsf{stmt}(n) = \texttt{call}^k \ e_0(e_1, \ldots, e_m), \\ \rho & \text{otherwise.} \end{cases}$$

Memory stores, labels, jumps, conditional jumps, and returns do not define ordinary temporaries in this analysis. A call is assumed to overwrite its ABI return temporaries with unknown values.

Instead of transferring on nodes, $U(n)$ transfers on edges and keets the same value within a single node. Thus the transfer function for unreachability $U(n)$ is defining the value of its successor nodes:

$$\forall n' \succeq n \qquad U(n') = \begin{cases} U(n) & \mathsf{stmt}(n) = \texttt{cjump} \ e \ L_t \text{ and } n = n_{if}(n) \text{ and } [\![e]\!]_\rho \neq \mathsf{Const}(0) \\ U(n) & \mathsf{stmt}(n) = \texttt{cjump} \ e \ L_t \text{ and } n = n_{next}(n) \text{ and } [\![e]\!]_\rho \neq \mathsf{Const}(z \neq 0) \\ \mathsf{true} & \mathsf{stmt}(n) = \texttt{cjump} \ e \ L_t \text{ and otherwise} \\ U(n) & \text{otherwise} \end{cases}$$

10.4 Detailed worklist algorithm

The worklist algorithm for conditional constant propagation is:

1. For every node n, initialize

   $$\mathsf{in}(n) := (true, \rho_\top), \qquad \mathsf{out}(n) := (true, \rho_\top).$$

   For start node s, initialize

   $$\mathsf{in}(n) = \mathsf{out}(n) = (false, \{_\mathsf{ARG}_1 \mapsto \mathsf{NAC}, \ldots, _\mathsf{ARG}_k \mapsto \mathsf{NAC}\}),$$

2. Initialize a FIFO worklist with all reachable nodes.

3. While the worklist is nonempty:

   (a) remove one node n from the worklist;

(b) calculate $U(n)$ by meeting all $U(n')$ coming from its predecessor nodes;

(c) if $U(n)$ is true, skip this node;

(d) compute

$$\mathsf{in}_{\mathrm{new}}(n) := \bigsqcap_{p \in \mathsf{pred}(n)} \mathsf{out}(p)$$

(e) compute

$$\mathsf{out}_{\mathrm{new}}(n) := (U(n), \mathsf{CTransfer}_n(\mathsf{in}_{\mathrm{new}}(n)))$$

(f) if either $\mathsf{in}_{\mathrm{new}}(n) \neq \mathsf{in}(n)$ or $\mathsf{out}_{\mathrm{new}}(n) \neq \mathsf{out}(n)$, then:

(i) set $\mathsf{in}(n) := \mathsf{in}_{\mathrm{new}}(n)$ and $\mathsf{out}(n) := \mathsf{out}_{\mathrm{new}}(n)$;

(ii) for every $s \in \mathsf{succ}(n)$, add s to the worklist if it is not already present.

The lattice has finite height for each temporary: a value can move only from Undef to a constant and then possibly to NAC. Since executable edges are only added and never removed, the algorithm terminates.

10.5 Expression rewriting

Given an environment ρ, define constant rewriting recursively:

$$\begin{aligned}
\mathsf{CRewrite}_\rho(z) &= z, \\
\mathsf{CRewrite}_\rho(t) &= \begin{cases} z & \rho(t) = \mathsf{Const}(z), \\ t & \text{otherwise}, \end{cases} \\
\mathsf{CRewrite}_\rho(\&L) &= \&L, \\
\mathsf{CRewrite}_\rho([e]) &= [\mathsf{CRewrite}_\rho(e)], \\
\mathsf{CRewrite}_\rho(e_1 \ op \ e_2) &= \begin{cases} z & [\![e_1 \ op \ e_2]\!]_\rho = \mathsf{Const}(z), \\ \mathsf{CRewrite}_\rho(e_1) \ op \ \mathsf{CRewrite}_\rho(e_2) & \text{otherwise}. \end{cases}
\end{aligned}$$

The rewrite folds only expressions whose abstract value is definitely constant. Memory loads are not folded, but their address expressions may be simplified.

Statement rewriting is defined by replacing temporaries only in use positions:

$$\begin{aligned}
\mathsf{CRewrite}_\rho(x \leftarrow e) &= x \leftarrow \mathsf{CRewrite}_\rho(e), \\
\mathsf{CRewrite}_\rho([e_1] \leftarrow e_2) &= [\mathsf{CRewrite}_\rho(e_1)] \leftarrow \mathsf{CRewrite}_\rho(e_2), \\
\mathsf{CRewrite}_\rho(\texttt{call}^k \ e_0(e_1, \ldots, e_m)) &= \texttt{call}^k \ e_0(\mathsf{CRewrite}_\rho(e_1), \ldots, \mathsf{CRewrite}_\rho(e_m)), \\
\mathsf{CRewrite}_\rho(\texttt{jump} \ e) &= \texttt{jump} \ e, \\
\mathsf{CRewrite}_\rho(\texttt{cjump} \ e \ L_t) &= \texttt{cjump} \ \mathsf{CRewrite}_\rho(e) \ L_t, \\
\mathsf{CRewrite}_\rho(\texttt{return} \ e_1, \ldots, e_r) &= \texttt{return} \ \mathsf{CRewrite}_\rho(e_1), \ldots, \mathsf{CRewrite}_\rho(e_r), \\
\mathsf{CRewrite}_\rho(L :) &= L : .
\end{aligned}$$

The left-hand side temporary of $x \leftarrow e$ is not rewritten, because it denotes a definition site.

10.6 Actual CFG transform for conditional constant propagation

After the fixed point is reached, the CFG is transformed as follows: For every executable real node $n \neq \mathsf{entry}$, let $\rho := \mathsf{in}(n)$. Replace $\mathsf{stmt}(n)$ by

$$\mathsf{CRewrite}_\rho(\mathsf{stmt}(n)).$$

After this transform, ordinary reachability cleanup and the control-flow eliminator would remove labels and jumps that became redundant. Also, this process neither deletes any node nor changes the node statement type, so no reconnection is needed.

11 Control Flow Eliminator

11.1 Overview

Remember that we have already implemented a trivial control flow analyzer on lowered IR before beginning the dataflow analysis on a real cfg. The trivial control flow analyzer does two things:

1. For each conditional jump statement which has both ifLabel and elseLabel, we rewrite it to:

$$\begin{aligned}\texttt{cjump}\ e\ L_t\ L_f := &\texttt{cjump}\ e\ L_t \\ &\texttt{jump}\ L_f\end{aligned}$$

2. For each jump statements which jumps to the label immediately after it, we remove this jump statement.

However, after dead code elimination and copy propagation, new dead jumps and dead labels may be generated, so we implement a new control-flow eliminator based on cfg analysis along with the other optimizations.

11.2 Setup

1. A jump node can be eliminated if:

$$\mathit{label2node}(L) = n_{i+1}$$

2. A cjump node can be eliminated when:

$$\begin{cases}\mathit{label2node}(L_t) = n_{i+1} \\ e = 0\end{cases}$$

3. A cjump node can be rewrited when:

$$\texttt{cjump}\ e\ L_t := \texttt{jump}\ L_t \qquad e \neq 0$$

4. A label node can be eliminated when:

$$\begin{cases}\mathit{pred}(n_i).\text{size}() = 0 \\ \mathit{pred}(n_i).\text{size}() = 1\ \&\ \mathit{pred}(n_i)[0] = n_{i-1}\end{cases}$$

11.3 Workflow

As the control flow eliminator is a relatively lighter process than dataflow analysis, everytime we call this eliminator, it should loop over the following workflow until no node elimination happens in the graph:

1. cjump elimination and cjump rewrite
2. jump elimination
3. call *reconnection*
4. label elimination
5. call *reconnection*

12 Combined Optimization Pipeline

As A straightforward implementation for one function is:

1. build the statement-level CFG from lowered IR;
2. call *reconnection*
3. remove unreachable nodes;
4. call *reconnection*
5. run available-copy analysis;
6. rewrite the CFG by copy propagation;
7. call *reconnection*
8. run conditional-constant-propagation analysis;
9. rewrite the CFG by constant propagation;
10. call *reconnection*
11. run live-variable analysis;
12. rewrite the CFG by dead-code elimination;
13. call *reconnection*
14. do recursive control-flow elimination
15. back to step 3 if any elimination of nodes happens in the graph compared to the last time
16. linearize the CFG back to lowered IR.